TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

TECHNICAL REPORT

Field	Details
Institute / University	Pimpri Chinchwad College of Engineering (PCCOE), Pune
Programme / Batch	B.Tech Computer Science and Engineering / 2027 Batch
Student Name	Bhagawati Kadam
PRN	123B1B154
Case Study ID (CS1–CS5)	CS2
Project Title	Functional Safety HARA and Safety Requirement Assistant

1. Problem Statement and Intended Users

In automotive functional safety (ISO 26262), a Hazard Analysis and Risk Assessment (HARA) lists the malfunctioning behaviours of an item, rates each hazardous event for severity (S), exposure (E) and controllability (C), derives an Automotive Safety Integrity Level (ASIL), and leads to safety goals and safety requirements that must be traceable. Done by hand in spreadsheets, this is slow, ratings vary between engineers, requirements are often written vaguely (no “shall”, no measurable value), and traceability gaps are found late.

The project provides an assistant for an Electric Power Steering (EPS) item that brings these steps into one controlled workflow. Retrieval-augmented generation (RAG) finds similar approved hazards and rules, a local LLM drafts hazards, safety goals and requirements, deterministic Python rules compute the ASIL, check requirement quality and detect traceability gaps, and an engineer approves every entry. The intended users are functional safety engineers, system and ECU engineers, and students learning HARA who cannot send item definitions to an external cloud service. The item and all ratings are synthetic and illustrative; the prototype does not claim ISO 26262 compliance.

2. Input Data / Knowledge Base

Item	Description
Sources	Synthetic content created for the project: item_definition.txt (EPS item, 9 blocks ITM-1 to ITM-9), hara_library.csv (14 approved hazardous events HE-001 to HE-014) and hara_rules.txt (23 rules, for example ASIL-1, ASIL-2, REQ-1 to REQ-5, SG-1, SG-2, TRC-1). No proprietary standard or OEM data is used.
Item functions and guidewords	Five functions: F1 Provide steering assist torque, F2 Measure driver steering torque, F3 Active return to centre, F4 Fault detection and driver warning, F5 Lane-keeping torque overlay. Seven guidewords, for example No / loss of function, Unintended activation, Too much / too little, Delay, Wrong direction, Stuck / intermittent.
Size and format	item_definition.txt: 9 blocks. hara_library.csv: 14 rows with function, malfunction, hazard, situation, S, E, C, ASIL and rationale. hara_rules.txt: 23 rules. HARA entries, goals and requirements are stored in SQLite (hara.db) and exported as CSV, JSON and Markdown.
Preprocessing / EDA done	Text is read and split into one chunk per item block, per library hazard and per rule, each with a stable ID. The stored ASIL of every library entry was checked against the rule ASIL-1 (14 of 14 consistent). Numerical EDA is not applicable.
Chunking strategy	Item blocks (9), library hazards (14) and rules (23) are one chunk each. Q&A retrieval returns the top 3 hazards, top 2 item blocks and top 3 rules.

Approved hazard library (synthetic, illustrative ratings):

ID	Fn	Malfunction	Situation	S	E	C	ASIL
HE-001	F1	Total loss of assist torque	Highway speed, driving through a curve	3	3	2	B
HE-002	F1	Unintended assist torque without driver input	Highway speed on a straight road	3	4	3	D
HE-003	F1	Excessive assist torque (over-assist)	High speed lane change	3	4	2	C
HE-004	F1	Insufficient assist torque (too little)	Parking or low speed manoeuvre	1	4	2	A
HE-005	F1	Assist torque applied in the wrong direction	Moderate speed, turn on an urban road	3	3	3	C

ID	Fn	Malfunction	Situation	S	E	C	ASIL
HE-006	F1	Delayed assist response	Evasive manoeuvre at medium speed	3	2	2	A
HE-007	F2	Torque sensor reports an erroneously high value	Urban driving at moderate speed	3	3	3	C
HE-008	F2	Torque sensor signal drops out intermittently	Rural road at medium speed	2	4	2	B
HE-009	F3	No active return to centre	Low speed turn at a junction or car park	1	4	1	QM
HE-010	F3	Excessive return torque	High speed on a curve	3	2	2	A
HE-011	F4	Warning lamp not activated when assist is lost	Any drive after a latent assist fault	2	3	3	B
HE-012	F4	False fault warning without a real fault	Normal driving	1	3	1	QM
HE-013	F5	Unintended lane-keeping torque overlay	Highway, straight road with lane keeping active	3	4	2	C
HE-014	F5	Overlay torque not released when the driver overrides	Highway obstacle avoidance with lane keeping active	3	3	3	C

3. Solution Architecture

Component	Choice (name, version, hosting)
Pre-trained model	Ollama running llama3.2:3b locally, temperature 0.1. Used for grounded Q&A and for drafting hazards, safety goals and requirements. No external API.
Embedding model	Sentence Transformers all-MiniLM-L6-v2 (sentence-transformers 6.1.0), run locally with normalised vectors. The model is downloaded once on first run.
Vector store (FAISS / ChromaDB)	FAISS (faiss-cpu 1.15.1): three local inner-product indexes for item blocks (9), approved hazards (14) and rules (23).
RAG pipeline / AI-agent design	Query or hazard description → embed → retrieve top-3 hazards, top-2 item blocks, top-3 rules → prompt builder (context marked untrusted, must cite IDs, must not decide the ASIL, reply "Not found in supplied context." if evidence is missing) → local LLM → draft with citation check. No autonomous agent; nothing is approved automatically.
Service layer and UI	Streamlit interface with four tabs: Item & Q&A, Hazard analysis (HARA), Safety goals & requirements, Traceability & export. Python modules: rag.py (indexing and retrieval), llm.py (prompts, citation check), asil.py, reqcheck.py, traceability.py and db.py (SQLite).
Tools / deterministic rules	asil.py implements rule ASIL-1; reqcheck.py implements REQ-1 to REQ-5; traceability.py implements TRC-1. These decide ASIL, requirement quality and gaps; the LLM only drafts text.

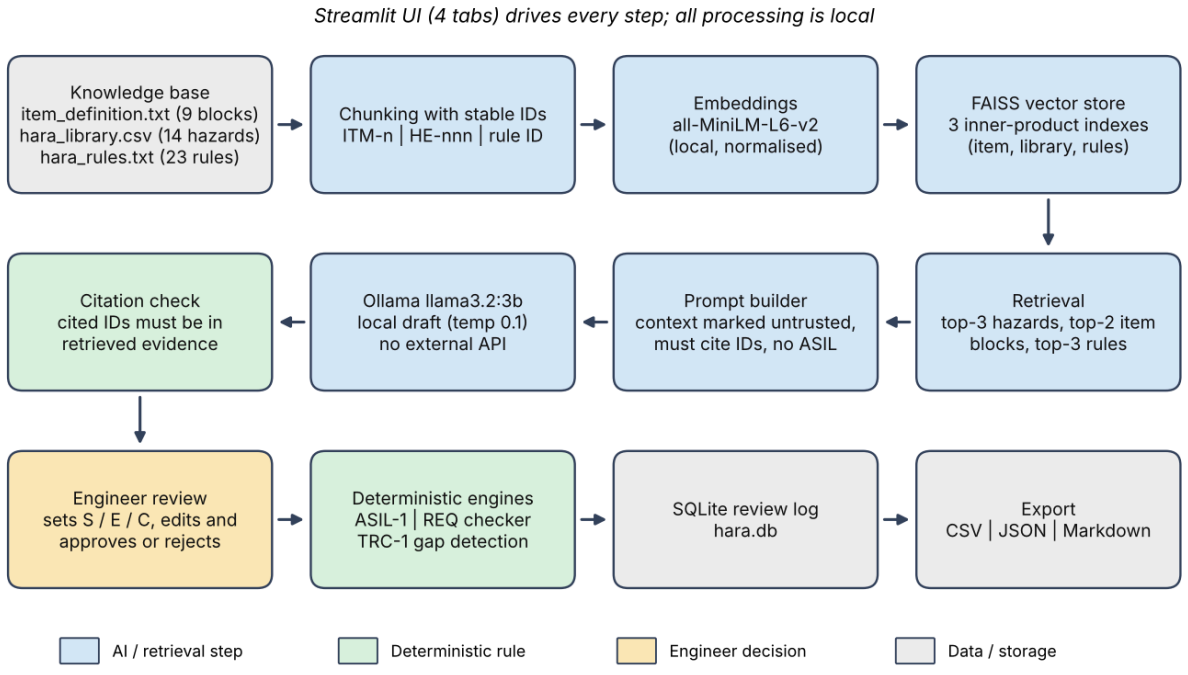


Figure 1: System architecture of the Functional Safety HARA and Safety Requirement Assistant (all processing is local).

4. Implementation and Configuration

The implementation follows one flow: knowledge-base indexing → hazard analysis → ASIL by rule → safety goal → FSR and TSR → traceability and export.

- **Start-up.** The item blocks, library and rules are chunked, embedded with all-MiniLM-L6-v2 and added to three FAISS indexes. The sidebar shows 9 item blocks, 14 library hazards, 23 rules, the model and the embedding model.
- **Item & Q&A.** The engineer asks about the item, the library or the rules. The retrieved evidence is passed to llama3.2:3b, which answers with cited IDs. A banner reports whether every cited ID was in the retrieved evidence.
- **Hazard analysis (HARA).** The engineer selects an item function, a guideword and an operating situation. The app lists similar approved library hazards with similarity scores and the LLM drafts the malfunction, hazard and suggested classes. S, E and C are pre-filled from the closest library entry and changed by the engineer; the ASIL is computed by rule ASIL-1 (sum S+E+C: 10 gives D, 9 C, 8 B, 7 A, 6 or less QM; any class 0 gives QM). The engineer then approves or rejects the hazardous event.
- **Safety goals & requirements.** For an approved hazardous event the LLM drafts a safety goal with a safe state and a fault tolerant time interval (FTTI); for a safety goal or FSR it drafts an FSR or TSR. The requirement checker flags missing "shall" (REQ-1), missing measurable value with unit (REQ-3), and missing ASIL, parent, allocation or verification method. Approval is blocked while issues remain.
- **Traceability & export.** The matrix links hazard → safety goal → FSR → TSR; rule TRC-1 reports gaps (NO_GOAL, NO_FSR, NO_TSR, NO_VERIFICATION, ASIL_MISMATCH) and a completeness percentage. The HARA worksheet, matrix and report are exported as CSV, JSON and Markdown. A "Load demo HARA data" button loads synthetic entries marked "demo seed", including a chain without a TSR so that the gap check can be demonstrated.

Prompt-injection and grounding measures: the system prompt marks all retrieved text as untrusted data, requires IDs as citations, forbids the model from deciding the ASIL and tells it to answer "Not found in supplied context." when evidence is missing. The run uses a Python virtual environment; dependencies are in requirements.txt and the model and prompt configuration is documented in Model_Prompts_Config/model_config.md.

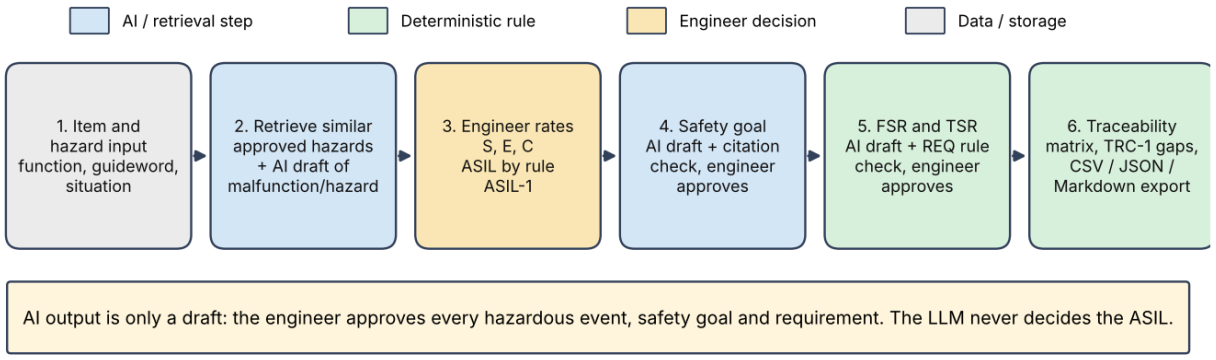


Figure 2: End-to-end workflow from hazard input to traceability export, showing AI, rule and engineer steps.

5. Evaluation Steps and Results

Evaluation is scripted in Code/run_eval.py (add --llm for AI drafts) and the output is saved in Evaluation_Results/ (eval_summary.json, retrieval_eval.csv, requirement_checker_eval.csv and llm_samples.json). The ASIL anchor cases, requirement-checker cases, injected traceability gaps and retrieval questions were written for this project with Claude's help; they are a small controlled test set, not an independent benchmark. Tests 6–10 were run in the Streamlit app.

#	Test question / scenario	Expected answer / behaviour	Result (Pass/Fail, notes)
1	ASIL rule on 23 reference S/E/C combinations (including S, E or C = 0)	Rule output equals the reference ASIL	Pass – 23 of 23 correct
2	Stored ASIL of the 14 library hazards checked against rule ASIL-1	All entries consistent	Pass – 14 of 14 consistent
3	Requirement checker on 10 requirements (3 compliant, 7 defective: vague wording, two requirements in one, missing allocation, verification, ASIL or parent)	Compliant accepted, defective flagged	Pass – 10 of 10 correct
4	Traceability check with 5 injected gaps (NO_GOAL, NO_FSR, NO_TSR, NO_VERIFICATION, ASIL_MISMATCH)	All gaps found, no false alarm	Pass – 5 of 5 detected, 0 false alarms; completeness 78.3% on the test set
5	Six retrieval questions with an expected hazard ID (e.g. "Warning lamp does not light when the power steering assist fails" → HE-011)	Expected ID in top-3 retrieved hazards	Pass – 6 of 6 in top-3; 5 of 6 at rank 1 (Q1: HE-002 0.54 ranked above expected HE-001 0.51)
6	Q&A: "How is the ASIL determined?"	Cites rule ASIL-1 with the correct S+E+C thresholds	Pass – answer cites ASIL-1 with correct thresholds; citation check passed (Figures 5, 6)
7	HARA: F1, guideword "No / loss of function", situation "Highway speed, straight road"	Relevant similar hazards and a consistent draft	Partly correct – similar hazards HE-001 (0.77), HE-002 (0.76), HE-003 (0.75); the AI draft mixed the hazard text of HE-002 (vehicle steers by itself) into a loss-of-torque case and suggested E4. The engineer rating pre-filled S3 E3 C2 = ASIL B and the hazard text was corrected (Figures 7, 8)
8	Safety goal draft for HZ-01 (ASIL B, total loss of assist torque)	Goal, safe state and FTTI with valid citations	Partly correct – citations valid (ITM-2, SG-1, SG-2); but the editable goal box received only "Safety goal:" (parsing of the model reply), the safe state restates the function, and FTTI 500 ms is the model's suggestion, not derived (Figure 9)
9	FSR draft for goal SG-01	Requirement with "shall" and a measurable value; valid citations	Fail, correctly blocked – citation check flagged SG-01 as not in the retrieved evidence; checker reported REQ-1 ("shall" missing) and REQ-3 (no measurable value); Approve FSR disabled (Figure 10)
10	Traceability tab on demo seed data plus entries created in the app	Matrix, gaps and exports shown	Pass – 4 hazardous events, 3 goals, 3 requirements, completeness 82.4%; gaps NO_GOAL (HZ-04), NO_FSR (SG-03), NO_TSR (FSR-02) listed; CSV/JSON/Markdown export available (Figures 11, 12)

Metric	Score / observation
ASIL rule accuracy	1.0 on 23 reference combinations (author-written reference values).
Library ASIL consistency	14 of 14 entries.
Requirement checker accuracy	1.0 on 10 cases (3 compliant, 7 defective).
Traceability gap detection	5 of 5 injected gaps detected, 0 false alarms.
Retrieval hit rate	Top-3: 1.0 (6 of 6 questions). Top-1: 0.833 (5 of 6).
Citation accuracy (AI drafts)	Safety goals: 3 of 3 drafts cited only retrieved IDs (run_eval.py --llm). FSR draft in the app cited SG-01, which was not retrieved, and was flagged.
Requirement quality of AI drafts	FSR drafts passing the requirement checker: 0 of 3 (run_eval.py --llm). The checker blocked all three.
Answer relevance / faithfulness	Manual review of sample outputs (tests 6–9). No numeric faithfulness score was collected.
Latency	Retrieval time printed by run_eval.py: 0.009 s to 1.033 s per question (the first call includes warm-up). End-to-end LLM response time was not formally benchmarked; it depends on local inference.

```
(venv) Code % python run_eval.py --llm
ASIL RULE:      {'anchor_cases': 23, 'correct': 23, 'accuracy': 1.0}
LIBRARY:        {'entries': 14, 'asil_consistent': 14}
REQ CHECKER:    {'cases': 10, 'correct': 10, 'accuracy': 1.0}
TRACEABILITY:   {'injected_gaps': 5, 'detected': 5, 'false_alarms': 0, 'completeness_pct_on_test_set': 78.3}
Retrieval (expected | top-3 retrieved with score | hit in top-3)
  Q1  HE-001 | HE-002(0.54); HE-005(0.53); HE-001(0.51) | True
  Q2  HE-002 | HE-002(0.42); HE-005(0.37); HE-001(0.37) | True
  Q3  HE-008 | HE-008(0.42); HE-007(0.31); HE-011(0.23) | True
  Q4  HE-011 | HE-011(0.63); HE-007(0.41); HE-008(0.39) | True
  Q5  HE-014 | HE-014(0.62); HE-013(0.62); HE-003(0.44) | True
  Q6  HE-009 | HE-009(0.54); HE-010(0.45); HE-005(0.36) | True
RETRIEVAL HIT RATE (top-3): 1.0   (top-1): 0.833
LLM DRAFTS:     {'safety_goals': 3, 'citations_valid': 3, 'fsr_drafts': 3, 'fsr_pass_checker': 0}
Saved results to Evaluation_Results
```

Figure 3: run_eval.py --llm output – rule accuracy, traceability gaps, retrieval hit rate and AI draft checks (layout condensed, values unchanged).

6. Responsible AI Measures

Area	Measure adopted in this project
Grounding and citations	Prompts restrict answers to retrieved evidence and require IDs as citations. An automatic citation check flags cited IDs that were not retrieved (Figure 10), and the evidence panel shows each source with its similarity score.
Privacy	The LLM, embeddings, vector index and database all run locally. No item data or prompt is sent to an external service after the one-time model downloads.
Security	Retrieved text is treated as untrusted in the prompt. AI output is advisory: nothing is approved, saved or executed automatically.
Human oversight / review	The ASIL is computed by rule, not by the model. An engineer sets S, E and C and approves or rejects every hazardous event, safety goal and requirement; the requirement checker blocks approval of vague or unmeasurable requirements.
Data	Only synthetic item data and illustrative ratings are used, and demo records are marked “demo seed”. The prototype does not claim ISO 26262 compliance; ratings need review by a qualified functional safety engineer before any real use.

7. Innovation Highlights

- Separates judgement from calculation: the LLM drafts text, while the ASIL, the requirement quality check and the traceability gaps are computed by deterministic rules.
- Automatic citation check and a requirement-quality gate catch weak AI output before it can be approved, as shown by the blocked FSR draft.
- Covers the full chain hazard → ASIL → safety goal → FSR → TSR with gap detection (TRC-1) and export, not only text generation.
- Runs fully offline with a small local model, suitable for item data that cannot leave the machine.

8. Screenshots, Logs and Evidence

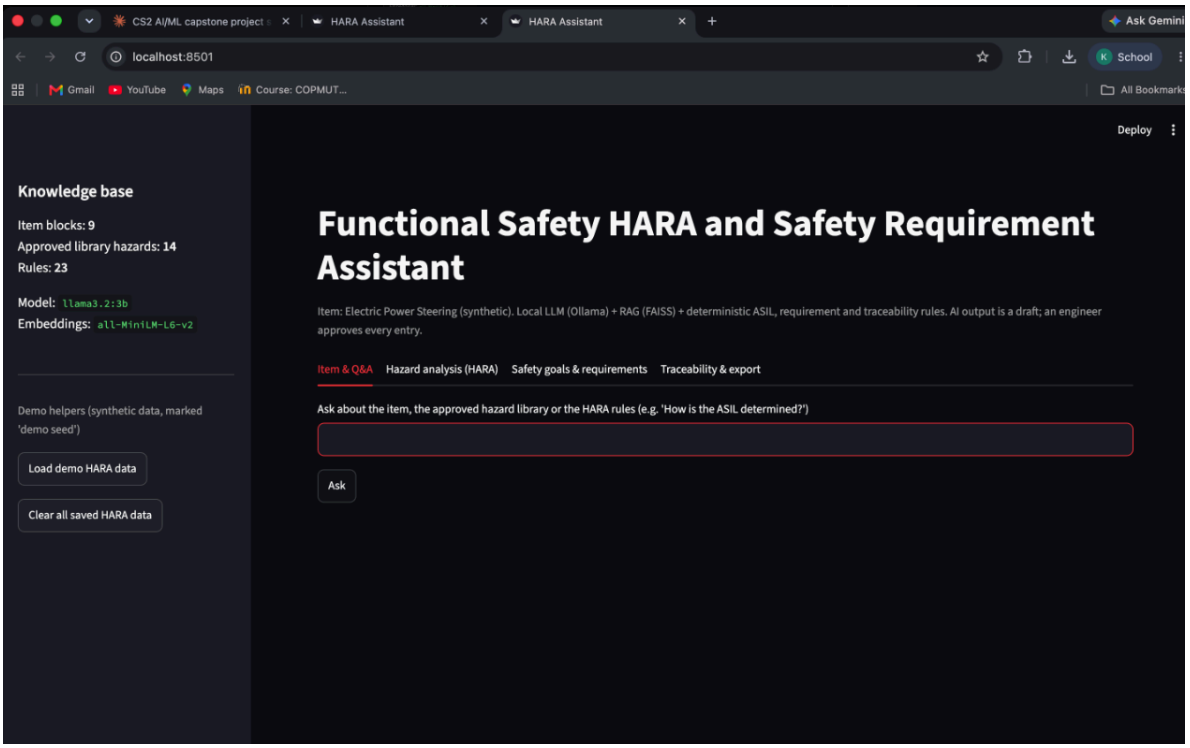


Figure 4: Home screen – knowledge base counts (9 item blocks, 14 library hazards, 23 rules), model and embeddings, and the four tabs.

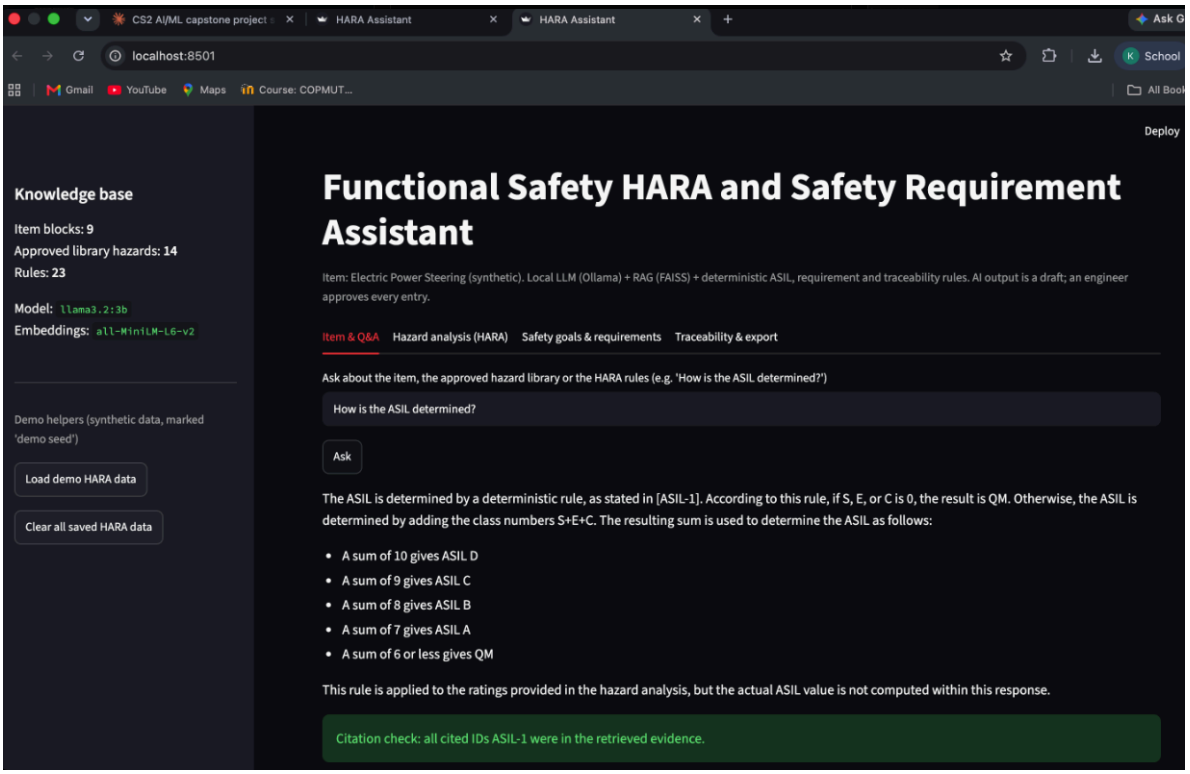


Figure 5: Item & Q&A – answer to "How is the ASIL determined?" citing ASIL-1, with a passed citation check.

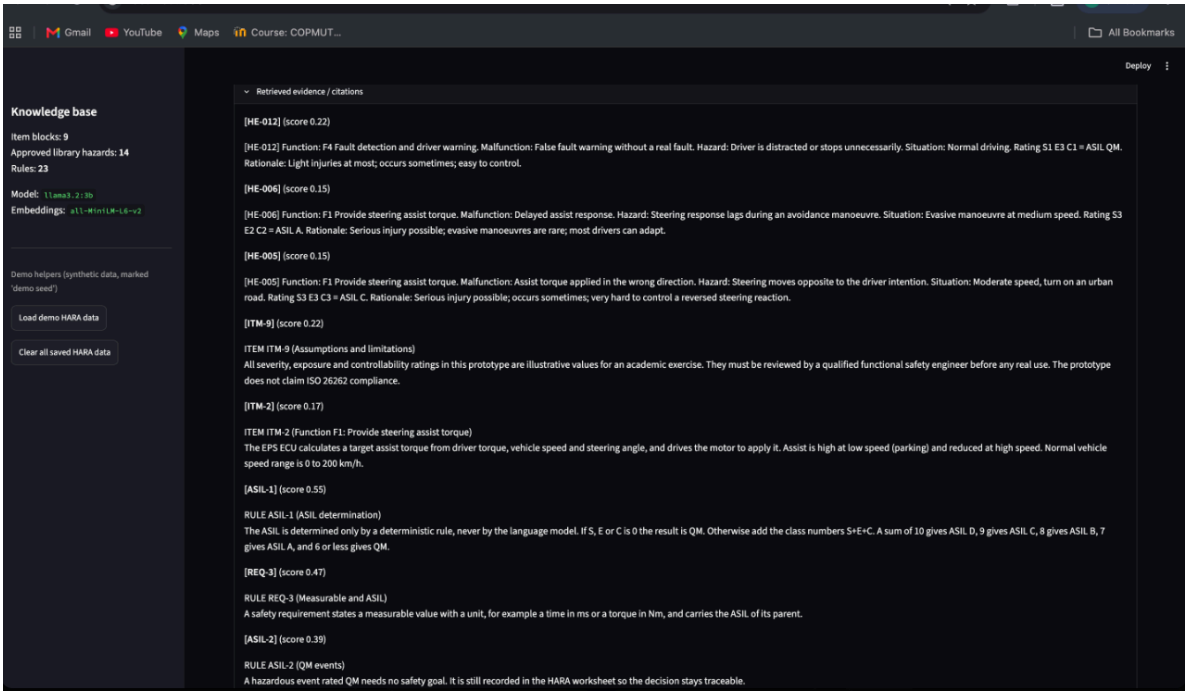


Figure 6: Retrieved evidence – library hazards, item blocks and rules with similarity scores.

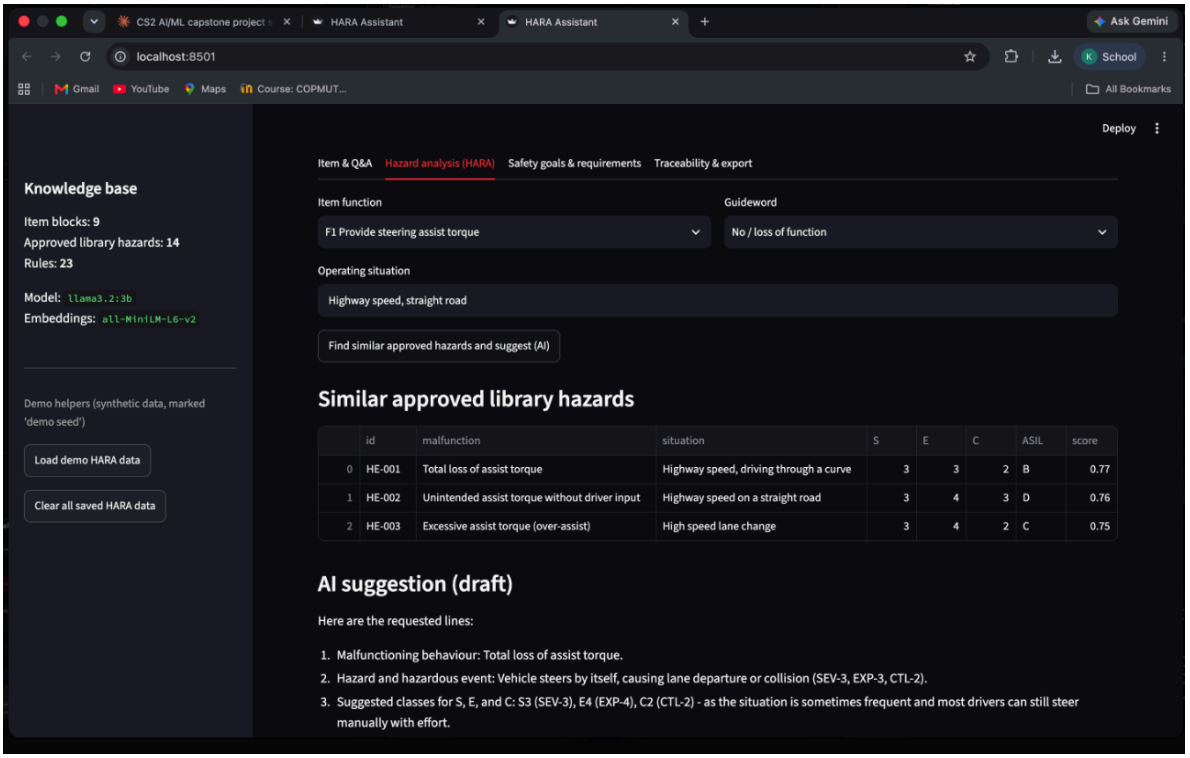


Figure 7: Hazard analysis – similar approved hazards and the AI suggestion (draft) for F1 / No or loss of function / Highway speed, straight road.

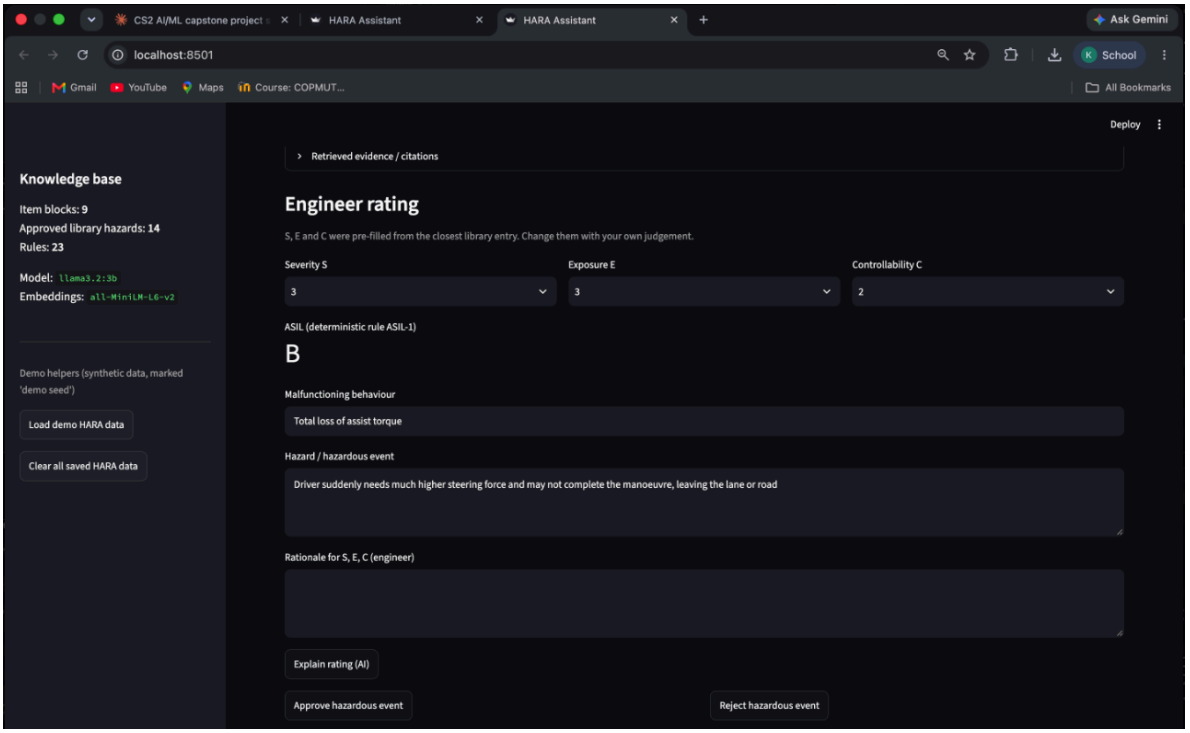


Figure 8: Engineer rating – S, E, C pre-filled from the closest library entry, ASIL B computed by rule ASIL-1, and Approve / Reject buttons.

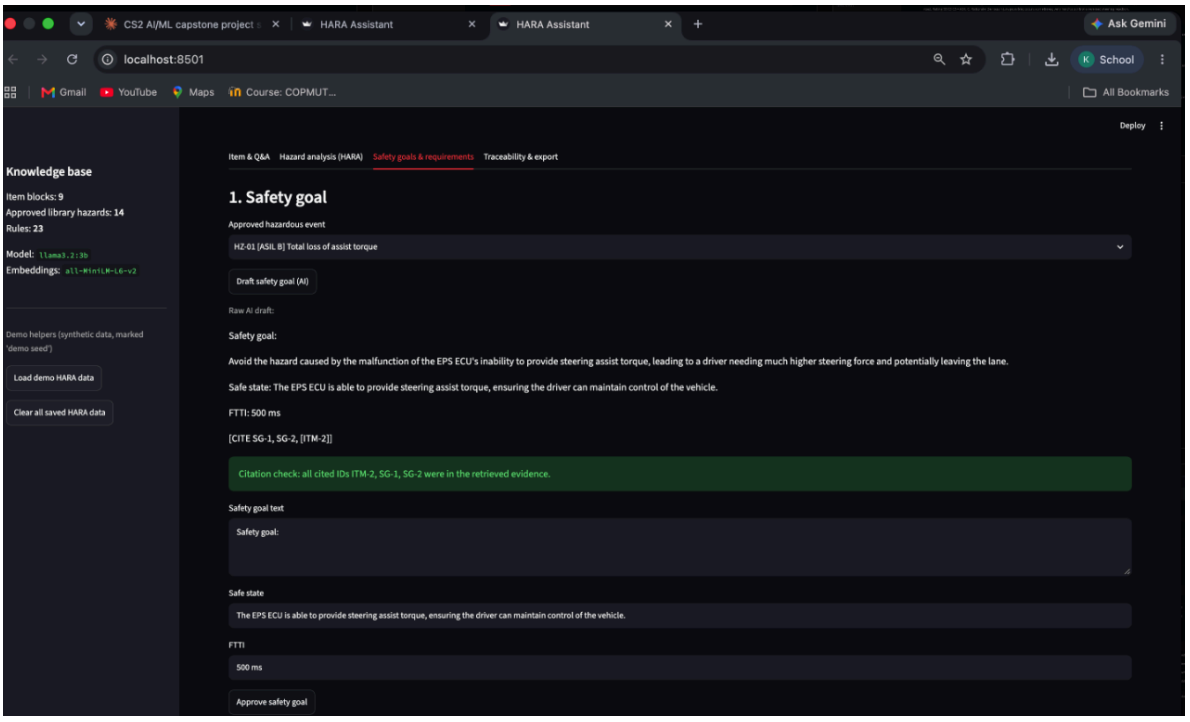


Figure 9: Safety goal – raw AI draft with a passed citation check, and editable goal, safe state and FTTI.

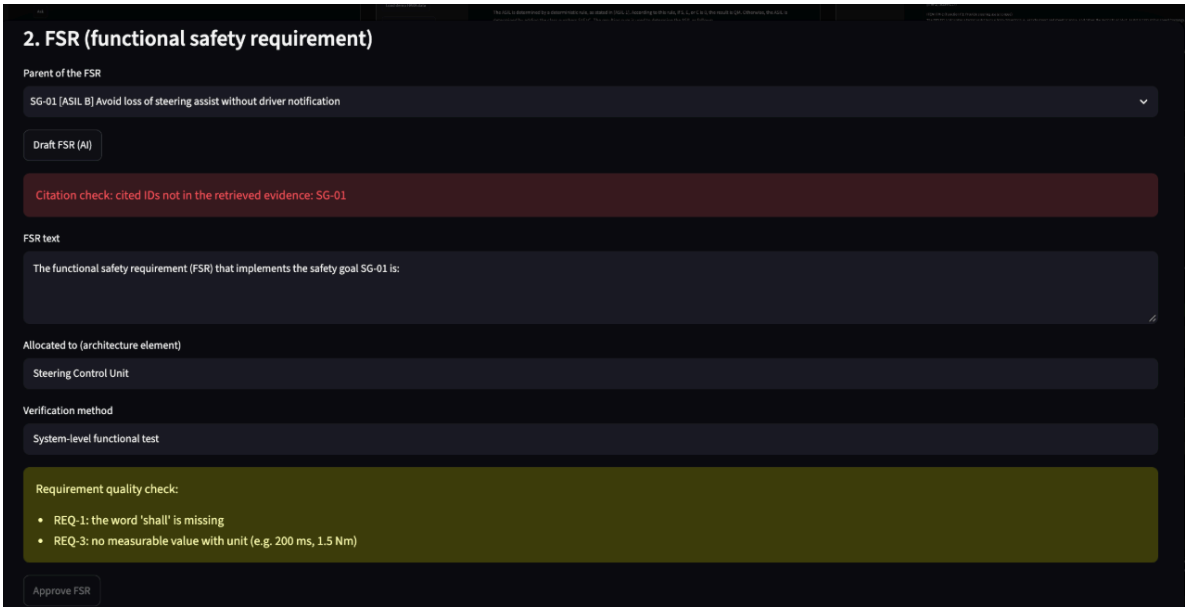


Figure 10: FSR draft – citation check flags SG-01, requirement quality check reports REQ-1 and REQ-3, Approve FSR disabled.

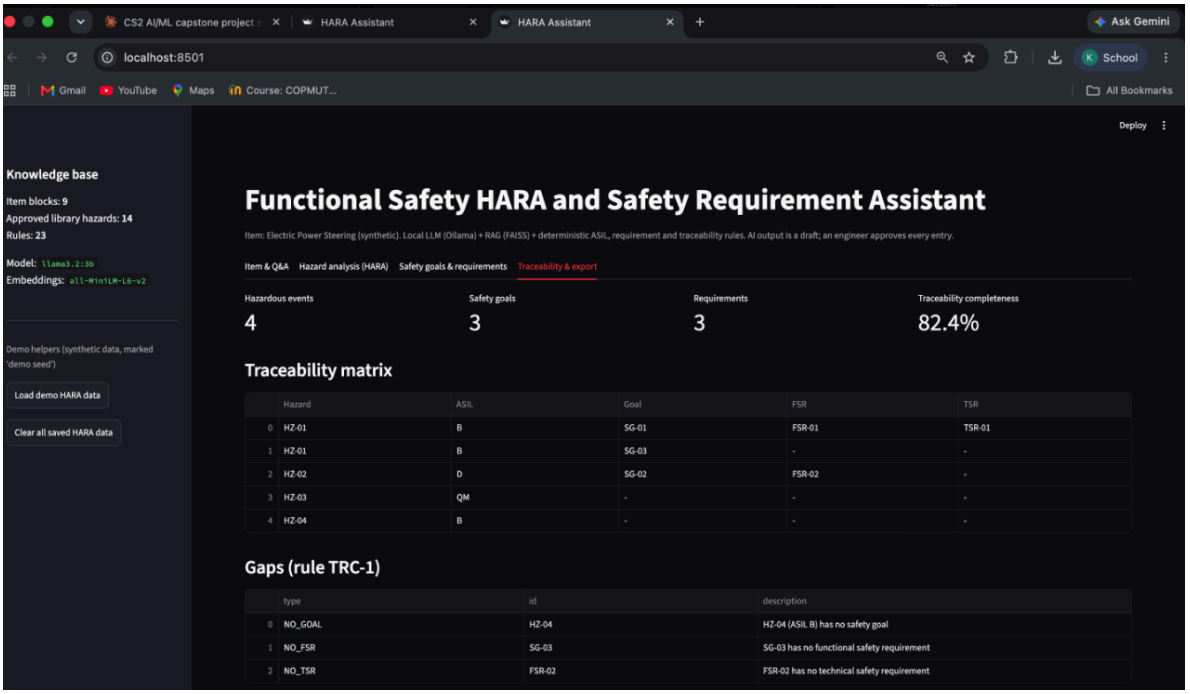


Figure 11: Traceability & export – counts, completeness 82.4%, traceability matrix and gaps (rule TRC-1).

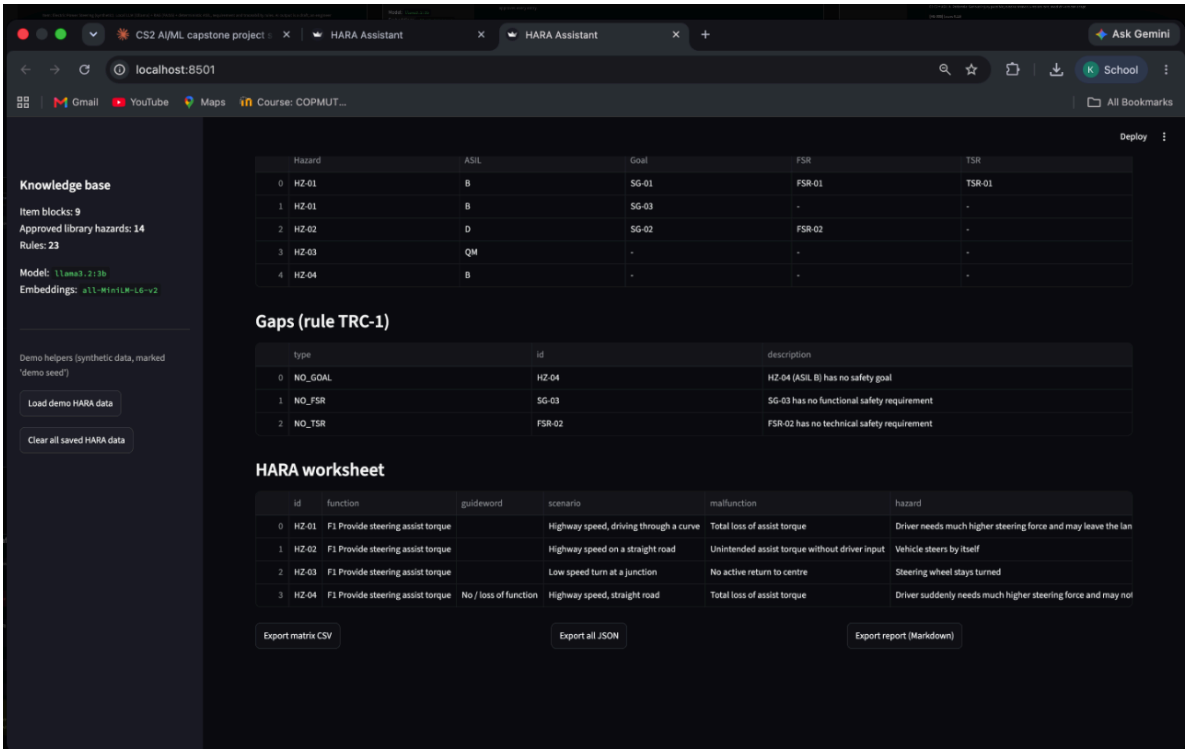


Figure 12: Gaps, HARA worksheet and export buttons (matrix CSV, all JSON, report Markdown).

9. Reflection and Conclusions

The project shows a practical way to support HARA and safety-requirement writing with AI without handing the decision to the model. Retrieval grounds the suggestions in approved hazards and rules, the ASIL, requirement checks and traceability gaps are deterministic, and the engineer approves every entry. All scripted checks passed on the synthetic test sets: 23 of 23 ASIL cases, 10 of 10 requirement cases, 5 of 5 injected gaps and 6 of 6 retrieval questions in the top 3.

The results are limited, and the 1.0 scores should not be read as general accuracy. The knowledge base is synthetic and small (14 library hazards, so the top 3 already covers a fifth of it), and the reference values and test cases were written by the author with AI assistance, so they show that the implementation matches its own specification, not that it generalises. Ratings are illustrative.

The 3-billion-parameter model is also not reliable enough to be trusted without review. None of the three FSR drafts passed the requirement checker (missing “shall” or measurable values); in the app the FSR draft cited a goal ID that was not in the evidence; the safety-goal draft was not parsed into the editable box; the proposed safe state restated the function and the FTTI was not derived; and one hazard draft mixed text from a different library entry. The embedding model also did not separate “driving through a curve” from “straight road”, ranking HE-001 above HE-002 for a straight-road query. These are the reasons the AI output is a draft and the quality gates and engineer approval are mandatory.

Next steps: fix the safety-goal parser, give the model requirement templates (“The … shall … within … ms”) and a repair loop driven by the checker output, test with a larger labelled hazard library and answer-correctness scoring, use a stronger local model, have a functional safety engineer review the ratings, and add Docker and CI for reproducible runs.

10. AI-Tool Usage Declaration

AI tool / model	Purpose	Artifact affected	Verified by Student (Y/N)	Remarks / Notes
Claude (Anthropic)	Coding assistance, generation of the synthetic knowledge base and test cases, debugging, and drafting of the synopsis and technical report.	Python modules, synthetic knowledge base, evaluation cases, project documentation, synopsis and technical report.	Y	Student ran the code and evaluation on a local machine and reviewed the affected artifacts.
Ollama / llama3.2:3b	Local LLM used by the application for Q&A and for drafting hazards, safety goals and requirements.	RAG/AI modules and generated draft content.	Y	Student reviewed the model output; it can be imprecise and requires engineer review.

11. Student Declaration

I confirm that this report represents my own understanding and contribution, that the work follows the approved case-study scope, and that I can explain every significant part of the implementation.

Signature:		Date:	07-10-2026
Name:	Bhagawati Kadam	PRN:	123B1B154